Elliptic Curve Cryptography in Practice

From the ECDLP to TLS, Signal, and the post-quantum transition

Diellart Mulolli (230756200035)

Code and LaTeX source: https://github.com/Diellart-Mulolli/ecc-lab

October 13, 2026

Introduction

Elliptic curve cryptography (ECC) is today the default public-key technology on the Internet: almost every TLS 1.3 handshake, every Signal and WhatsApp message, every SSH login with a modern key and every Bitcoin transaction relies on an elliptic curve. Its attraction is that the elliptic-curve discrete logarithm problem (ECDLP) admits no known sub-exponential classical attack, so 256-bit keys give the same security as 3072-bit RSA keys. This report first works through the mathematics on a toy curve and compares ECC with RSA, then studies how the same curve is used for key agreement, signatures and encryption, analyses a real-world failure (the 2024 PuTTY P-521 key-recovery vulnerability), and finally discusses the threat of quantum computers and how a long-lived system should choose its mechanisms. Three small Python scripts were used to verify the calculations and to measure performance; they are published, together with the LaTeX source of this report and a running guide, at https://github.com/Diellart-Mulolli/ecc-lab. Appendix A lists the values chosen in each script and its exact output.

Foundations

Question 1 — The ECDLP

Curve (Stamp, Sec. 4.5.1 [1]): $E : y^2 \equiv x^3 + 2x + 1 \pmod 5$, base point $P = (0,1)$, $E = \{(0,1),(0,4),(1,2),(1,3),(3,2),(3,3),\infty\}$, so $\#E = 7$ (prime), and P has order $n = 7$.

(a) Computation of $2P$ (doubling, $x_1 = 0,\ y_1 = 1,\ a = 2$):

$$m = (3x_1^2 + a)(2y_1)^{-1} \bmod 5 = (3\cdot 0 + 2)\cdot 2^{-1} = 2\cdot 3 = 6 \equiv 1,$$
$$x_3 = m^2 - x_1 - x_2 = 1 - 0 - 0 = 1, \qquad y_3 = m(x_1 - x_3) - y_1 = 1\cdot(0-1) - 1 = -2 \equiv 3.$$

So $2P = (1,3)$. Check: $y^2 = 9 \equiv 4$ and $x^3 + 2x + 1 = 1 + 2 + 1 = 4$. ✓

Computation of $3P = 2P + P = (1,3) + (0,1)$ (addition):

$$m = (y_2 - y_1)(x_2 - x_1)^{-1} = (1-3)(0-1)^{-1} = (-2)(-1)^{-1} \equiv 3\cdot 4^{-1} = 3\cdot 4 = 12 \equiv 2,$$
$$x_3 = m^2 - x_1 - x_2 = 4 - 1 - 0 = 3, \qquad y_3 = m(x_1 - x_3) - y_1 = 2(1-3) - 3 = -7 \equiv 3.$$

So $3P = (3,3)$. Check: $y^2 = 9 \equiv 4$ and $27 + 6 + 1 = 34 \equiv 4$. ✓

(b) The private key was found by *exhaustive search*: compute $P, 2P, 3P, \ldots$ by repeatedly adding P and compare each result with Q: $2P = (1,3)$, $3P = (3,3)$, $4P = 3P + P = (3,2) = Q$. Hence $\boxed{k = 4}$. The search needed 3 group operations (one doubling and two additions). The script `q1_toy_curve.py` reproduces (a) and (b) exactly (Appendix A.1). Consistency check: $4P = -3P$ since $7P = \infty$, and indeed $(3,2) = -(3,3)$. The full cycle is $P = (0,1)$, $2P = (1,3)$, $3P = (3,3)$, $4P = (3,2)$, $5P = (1,2)$, $6P = (0,4)$, $7P = \infty$. Brute force needs on average $n/2$ operations; this is trivial for $n = 7$ but impossible for $n \approx 2^{256}$.

(c)

(i) With the *double-and-add* algorithm (the elliptic-curve analogue of square-and-multiply), d is processed bit by bit: one doubling per bit and one addition per 1-bit. For a 256-bit d this is about 255 doublings and on average ≈ 128 additions, i.e. fewer than 400 group operations. Constant-time implementations use the Montgomery ladder (one doubling and one addition per bit, $\approx 2 \times 255$ operations) and fixed-base precomputed tables or windowing (wNAF) to reduce the number of additions further. Computing $Q = dG$ therefore takes well under a millisecond.

(ii) The best known classical attack on a well-chosen curve is *Pollard's rho* (with van Oorschot–Wiener parallelisation). It needs about $\sqrt{\pi n/4} \approx 0.89\sqrt{n}$ group operations, i.e. $\approx 2^{128}$ for P-256. Generic algorithms such as baby-step giant-step have the same $O(\sqrt{n})$ time but also need $O(\sqrt{n})$ memory. This is fully exponential in the key length: 128-bit security.

(iii) For factoring and for the discrete logarithm in $\mathbb{Z}_p^*$ there exist *sub-exponential* index-calculus algorithms, above all the General Number Field Sieve, with cost $L_N[1/3, 1.923] = \exp\left((1.923 + o(1))(\ln N)^{1/3}(\ln \ln N)^{2/3}\right)$. They exploit extra structure (smooth integers, a factor base of small primes). Elliptic-curve points have no notion of "small" or "smooth" elements, so index calculus does not apply to well-chosen curves and only generic $O(\sqrt{n})$ attacks remain. Because security grows linearly with the ECC key length (half the bits) but only like the cube root of the RSA modulus size, ECC reaches the same security with keys roughly 12 to 30 times shorter.

(d) *Technical reason:* generating a secure curve is hard and easy to get wrong. One must count the points (Schoof/SEA), ensure that the group order is a large prime (times a small cofactor) and exclude weak classes (anomalous curves with $\#E = p$, curves with small embedding degree vulnerable to the MOV/Frey–Rück reduction, singular curves, weak twists). Named curves such as P-256 or Curve25519 [2], [3] have been analysed publicly for many years and can be validated against a single reference. *Practical reason:* interoperability and efficiency. Both peers just send a curve identifier (e.g. `x25519`, `secp256r1`) instead of full parameters, and implementations can use highly optimised, constant-time, formally verified or hardware-accelerated code for a few fixed curves. Accepting arbitrary parameters has also led to vulnerabilities (CurveBall, CVE-2020-0601, where Windows accepted attacker-chosen explicit curve parameters).

Question 2 — ECC versus RSA

(a)

Security strength	Symmetric	RSA modulus (bits)	ECC key (bits)
112	3TDEA*	2048	224–255
128	AES-128	3072	256–383
192	AES-192	7680	384–511
256	AES-256	15360	512+

Table 1: Comparable security strengths, adapted from [4] (Table 2). *3TDEA is deprecated and disallowed after 2023.

(b) At 128-bit security: $3072/256 = 12$. At 256-bit security: $15360/512 = 30$. The gap grows because the cost of breaking ECC is fully exponential ($\approx 2^{\ell/2}$ for an ℓ-bit key), so doubling security only needs a doubling of the key size, while the GNFS is sub-exponential: the RSA modulus has to grow roughly with the *cube* of the security level (up to logarithmic factors). Doubling the security from 128 to 256 bits multiplies the RSA modulus by 5 (3072 to 15360 bits) but the ECC key only by 2 (256 to 512 bits).

(c) Measurements made with `q2_benchmark.py` (Python `cryptography` 41, OpenSSL backend, one laptop core; average milliseconds per operation; settings and raw output in Appendix A.2) and sizes from the standards [5], [6]:

Scheme	Key generation	Signing	Verification	Public key	Signature
RSA-3072 (PSS)	813 ms (slowest)	1.39 ms	**0.09 ms**	384 B modulus (422 B DER)	384 B
ECDSA P-256	0.06 ms	0.05 ms	0.11 ms	64 B raw (65 B uncompr., 33 B compr.)	64 B raw (69–72 B DER)
Ed25519	0.07 ms	0.06 ms	0.16 ms	32 B	64 B

Table 2: RSA-3072 versus ECDSA P-256 versus Ed25519 (128-bit security).

Key generation in RSA is four orders of magnitude slower (two random 1536-bit primes must be found), signing is ≈ 25 times slower, and the keys and signatures are 6–12 times larger. *RSA is faster at verification*: verification computes $s^e \bmod N$ with the small public exponent $e = 65537 = 2^{16} + 1$, i.e. only 17 modular multiplications, whereas ECDSA/EdDSA verification requires a double scalar multiplication with two full-size 256-bit scalars (several hundred point operations).

(d) For the battery-powered sensors I recommend **Ed25519** (or ECDSA P-256 if the hardware only offers a NIST-certified accelerator). The signer is the constrained party: Ed25519 signing is cheap, needs no random number generator per signature (no Sony-style nonce failure on a cheap device), is easy to implement in constant time, and produces 64-byte signatures, which saves radio energy compared with 384-byte RSA signatures. The cloud verifies $50\,000/60 \approx 833$ signatures per second; at ≈ 0.15 ms per verification this uses about 13% of one CPU core, and Ed25519 also supports batch verification. The answer changes partly in the opposite scenario: if a powerful server signs and millions of small devices only verify, RSA's very cheap verification (small e) becomes an advantage and the large signature size matters less, so RSA-3072 is a reasonable choice (e.g. for firmware images), although an Ed25519 verifier with a 32-byte key is also feasible on most microcontrollers and much smaller in code and key storage.

Mechanisms and Protocol Scenarios

Question 3 — ECDH, ECDSA, EdDSA, and ECIES

(a)

Mechanism	Security goal	Inputs / outputs	Fresh randomness?	Standard	Real system
ECDH / X25519	Key agreement: a shared secret over a public channel (confidentiality, forward secrecy when ephemeral)	In: own private scalar, peer public point. Out: shared point → KDF → session keys	Yes, for each ephemeral key pair	RFC 7748, NIST SP 800-56A	TLS 1.3, Signal, WireGuard, SSH
ECDSA	Authenticity, integrity, non-repudiation	Sign: d, message $\to (r, s)$. Verify: Q, message, $(r, s) \to$ valid/invalid	Yes: secret nonce k per signature (unless RFC 6979)	FIPS 186-5, SEC 1, ANSI X9.62	X.509/TLS certificates, Bitcoin, Ethereum
EdDSA / Ed25519	Authenticity, integrity, non-repudiation	Sign: secret key, message $\to (R, S)$ (64 B). Verify: public key, message, signature → valid/invalid	No: nonce is derived deterministically	RFC 8032, FIPS 186-5	OpenSSH, Signal (XEdDSA), Tor onion services, OpenBSD signify
ECIES / HPKE	Public-key encryption: confidentiality and integrity to a recipient's public key	In: recipient public key, plaintext, (info, AAD). Out: encapsulated ephemeral key `enc` + AEAD ciphertext	Yes: a fresh ephemeral key per message	SEC 1, ISO 18033-2; RFC 9180 (HPKE)	TLS ECH, MLS (RFC 9420), Apple iMessage/PQ3, Cloudflare ODoH

(b) The statement is inaccurate because there is no practical elliptic-curve analogue of the RSA trapdoor permutation $m \mapsto m^e \bmod N$: an EC public key $Q = dG$ is a point, and there is no efficient way to map an arbitrary message into a point and back with a trapdoor. (The textbook "EC-ElGamal" exists but needs message encoding into points, doubles the ciphertext size and is malleable, so it is not used.) ECC itself only provides *key agreement* and *signatures*. In practice, a message is encrypted with a *hybrid* construction, ECIES or HPKE [7] (mode `base`, e.g. `DHKEM(X25519, HKDF-SHA256)` with AES-128-GCM):

1. **KEM (key encapsulation):** the sender generates a fresh ephemeral key pair $(e, E = eG)$ and computes $Z = e\,Q_{\text{recipient}}$ (ECDH). E is sent as `enc`.
2. **KDF (HKDF):** Z, E, Q and context information are turned into a uniformly random symmetric key and nonce.
3. **DEM / AEAD (AES-GCM or ChaCha20-Poly1305):** encrypts and authenticates the actual data.

The recipient computes $Z = d\,E$, derives the same key and decrypts. ECC handles only the key; the symmetric cipher handles the data, which also allows arbitrarily long messages.

(c) Both signatures use the same k, so $r = x(kG) \bmod n$ is the same. Then

$$s_1 = k^{-1}(h_1 + rd), \qquad s_2 = k^{-1}(h_2 + rd) \pmod{n}.$$

Subtracting eliminates the unknown d: $s_1 - s_2 = k^{-1}(h_1 - h_2)$, hence

$$\boxed{k = (h_1 - h_2)(s_1 - s_2)^{-1} \bmod n}, \qquad \boxed{d = (s_1 k - h_1)\, r^{-1} \bmod n}.$$

Both h_1, h_2 are hashes of public messages, so everything on the right-hand side is public. The reuse is visible directly: two signatures with an *identical r value* under the same key reveal that the same k was used (with a random 256-bit k, a repeated r would otherwise happen with negligible probability). I verified this with `q3_nonce_reuse.py` on P-256 (run with `--seed 2026`; all chosen values and the full output are in Appendix A.3): both signatures share $r =$ `0x45f4...0b8e` and are valid, also when checked with OpenSSL; the two formulas above return exactly the k and d that were used; and the recovered key forges a valid signature on a new message. In the Sony PS3 case this allowed fail0verflow to sign arbitrary software [8].

(d) *EdDSA* (RFC 8032) hashes the 32-byte secret key with SHA-512; the second half of the output is a secret `prefix`. The per-signature value is $r = \text{SHA-512}(\texttt{prefix} \,\|\, M) \bmod \ell$ and $R = rB$. It is deterministic, secret and different for every message. *RFC 6979* keeps the ECDSA equations but replaces the random k by a value generated with HMAC-DRBG seeded by the private key x and $H(m)$, so the same message always gives the same signature and different messages give independent-looking nonces [9]. These designs remove the class of **randomness failures**: reused, predictable or biased nonces caused by a weak or broken RNG (Sony PS3, the 2013 Android `SecureRandom` Bitcoin thefts). Still possible are **implementation attacks**: side-channel attacks (timing, cache, power, EM, e.g. Minerva and TPM-Fail which leak nonce bits through non-constant-time scalar multiplication), **fault attacks** (deterministic signing is especially vulnerable: a single glitch while signing the same message twice leaks the key), and incorrect implementations of the derivation itself (PuTTY, Question 5). Lattice attacks on partially known nonces also remain possible if any nonce bits leak.

Question 4 — Alice, Bob, and Mallory

(a) Alice computes $K_A = aB = a(bG)$ and Bob computes $K_B = bA = b(aG)$. Since scalar multiplication is repeated addition in an abelian group, $a(bG) = (ab)G = (ba)G = b(aG)$, so $K_A = K_B = abG$. Mallory sees only A and B; computing abG from them is the elliptic-curve Diffie–Hellman problem. The x-coordinate is passed through a KDF (HKDF-Extract/Expand in TLS 1.3 [10], [11]) because (1) it is *not uniformly random*: it is an element of $\mathbb{F}_p$, only about half of all x-values occur on the curve, and it has algebraic structure, while AES keys must be uniform; (2) one secret must produce *several independent keys* (client/server keys, IVs, handshake versus application traffic keys, resumption secrets); (3) HKDF binds the key to its *context* (labels and the transcript hash), so keys cannot be reused across protocols or sessions; and (4) the length must match (e.g. 128 or 256 bits).

(b) Man-in-the-middle attack: Mallory chooses her own secrets m_1, m_2.

- Alice sends A; Mallory intercepts it and sends $M_1 = m_1 G$ to Bob as if it came from Alice.
- Bob sends B; Mallory intercepts it and sends $M_2 = m_2 G$ to Alice as if it came from Bob.
- Alice computes $K_1 = aM_2 = am_2G$ and Mallory computes the same $m_2 A$. Bob computes $K_2 = bM_1 = bm_1G$ and Mallory computes the same $m_1 B$.

At the end there are two keys: K_1 (Alice–Mallory) and K_2 (Mallory–Bob). Alice believes she shares K_1 with Bob, Bob believes he shares K_2 with Alice; Mallory decrypts, reads or modifies, and re-encrypts every message. Moving from a 2048-bit finite-field group to a 256-bit curve does not help, because the attack does not solve any discrete logarithm: Mallory simply performs two honest key exchanges. The problem is missing *authentication* of the public values, not the hardness of the underlying problem.

(c) In TLS 1.3 the key exchange (`key_share` in ClientHello/ServerHello) is authenticated by the server's **CertificateVerify** message. It contains a signature (e.g. ECDSA P-256 with SHA-256, RSA-

PSS, or Ed25519) over 64 bytes of `0x20`, the context string "`TLS 1.3, server CertificateVerify`", a zero byte, and the *transcript hash* of all handshake messages from ClientHello up to and including Certificate, which includes both key shares [10]. The signature is verified with the public key in the server's leaf *certificate*. The client trusts that key if the certificate chain leads to a root CA in its trust store, all signatures in the chain verify, the certificate is valid (dates, key usage), the hostname matches the Subject Alternative Name, and it is not revoked (and in browsers carries Certificate Transparency proofs). The `Finished` MAC then confirms that both sides derived the same keys. If Mallory replaces B, she cannot produce a valid signature over the modified transcript without the server's private key. *Signal* has no CAs: each user has a long-term identity key (Curve25519/Ed25519 via XEdDSA) and publishes a *signed prekey* and one-time prekeys to the server. The X3DH/PQXDH handshake mixes several DH computations involving the identity keys, so only the owner of the identity key can derive the session keys [12], [13]. Trust in the identity key is obtained by trust on first use plus optional out-of-band verification of *safety numbers* (comparing 60 digits or scanning a QR code); a change of identity key is shown to the user as a warning.

(d) No. TLS 1.3 only uses ephemeral (EC)DHE: the session keys came from abG, and a, b were deleted after the handshake. The stolen ECDSA key was used only to *sign*, so it does not help to compute abG; the recorded 2026 sessions remain confidential (*forward secrecy*). With the stolen key the attacker can *impersonate* the server in the future: run active man-in-the-middle attacks on new connections or set up a phishing server with the genuine certificate, until the certificate is revoked and the key replaced. (Exceptions: if session-ticket encryption keys were also stolen, resumed 0-RTT/PSK-only sessions may be exposed.) In TLS 1.2 with RSA key transport, the client encrypted the pre-master secret with the server's long-term RSA public key, so the stolen RSA key would decrypt every recorded pre-master secret and thus *all* past sessions; there is no forward secrecy. This is one reason TLS 1.3 removed RSA key transport.

Real-World Investigation

Question 5 — PuTTY NIST P-521 private-key recovery (CVE-2024-31497)

(a) Incident profile

Discovery and disclosure	Reported privately to the PuTTY developers before disclosure; publicly disclosed on 15 April 2024 together with the fixed release [14], [15].
Discovered by	Fabian Bäumer and Marcus Brinkmann, Ruhr University Bochum.
Affected products and scale	PuTTY 0.68–0.80 (2017–2024) and Pageant, and products embedding PuTTY code: FileZilla 3.24.1–3.66.5, WinSCP 5.9.5–6.3.2, TortoiseGit 2.4.0.2–2.15.0, TortoiseSVN 1.10.0–1.14.6. Only *NIST P-521* keys (`ecdsa-sha2-nistp521`) are affected, but every such key used with these tools over 7 years must be considered compromised. PuTTY is one of the most widely used SSH clients on Windows.
ECC mechanism	ECDSA signature generation, specifically the derivation of the per-signature nonce k (random number generation).
Identifier	CVE-2024-31497; PuTTY advisory `vuln-p521-bias` [14], [16].
Fix	PuTTY 0.81 (15 April 2024) switched all DSA/ECDSA nonce generation to RFC 6979; FileZilla 3.67.0, WinSCP 6.3.3 and TortoiseGit 2.15.0.1 shipped fixes. TortoiseSVN had no patch at disclosure, so users were advised to use Plink from PuTTY 0.81 for SVN over SSH. The fix cannot repair keys that already leaked; they must be revoked and regenerated.

(b) Technical root cause.

1. Since PuTTY first implemented DSA (around 2001, more than ten years before RFC 6979) it has generated k deterministically, because its author did not trust the entropy source of Windows: k was a SHA-512 hash of secret data derived from the private key together with a hash of the message, reduced modulo n.

2. This is fine for curves with $n < 2^{512}$, but for P-521 the order n has 521 bits, while SHA-512 produces only 512 bits. Reducing a 512-bit value modulo a 521-bit n changes nothing, so the **top 9 bits of every nonce are always zero**. The nonce is biased: it is not uniform in $[1, n-1]$.

3. Each signature then gives a linear relation $s_i k_i \equiv h_i + r_i d \pmod{n}$ with a known small bound $k_i < 2^{512}$. This is an instance of the *Hidden Number Problem*: with about 60 signatures, a lattice built from these equations and reduced with LLL/BKZ reveals d.

4. The attacker needs about 60 signatures and the public key. They can be collected by (i) a malicious or compromised SSH server to which the victim logs in (each login produces a signature on data the server knows), or (ii) passively from public data, e.g. Git commits signed through PuTTY/Pageant and published on GitHub.

The failed property is the **unpredictability** (uniformity) of the nonce, which leads to a failure of **authenticity**: the attacker obtains the **private key**, can log in to every server that trusts it (supply-chain attacks via Git/SSH access) and can forge signatures in the victim's name.

(c) Classification. The failure is an **implementation** error: a nonce-generation routine designed for smaller curves was reused for P-521 without checking that the hash length is at least the bit length of n. The ECDSA standard (FIPS 186-5) and RFC 6979 specify how to generate unbiased nonces; P-521 and the mathematics of elliptic curves are unaffected. The **ECDLP was not broken**: the attack never computes a discrete logarithm. It uses the extra information leaked by the biased nonces together with lattice reduction. Keys on the same curve generated by correct implementations (e.g. OpenSSH) remain secure.

(d) Response and lessons. *Vendors* coordinated disclosure: PuTTY, FileZilla, WinSCP and TortoiseGit released fixes on the disclosure date or shortly after (TortoiseSVN users were pointed to the patched Plink in the meantime), and the PuTTY advisory clearly stated that fixing the software is not enough. Linux distributions shipped patched packages. *Users* were told to remove the old P-521 public keys from every `authorized_keys` file and from GitHub/GitLab, generate new keys (preferably Ed25519), and re-sign if needed. *Standards*: no standard needed to be withdrawn; the incident reinforced RFC 6979 and the deterministic design of EdDSA. I consider the technical response adequate and fast (fixed on the day of disclosure, clear advisory), but the *key rotation* relies on each user, and signed Git commits remain public forever, so already-leaked keys cannot be "un-leaked". **Proposed control:** known-answer and statistical tests for nonce generation in the test suite, e.g. Project Wycheproof-style tests that sign many messages for each supported curve and check that the nonces use the full range of n (a check of the highest bits would have revealed the 9 zero bits immediately). As a process control, review rule: any reuse of a cryptographic routine for a new parameter set must check its length assumptions ($|H| \geq \lceil \log_2 n \rceil + 64$).

The Quantum Horizon and Design Decisions

Question 6 — ECC, Shor's algorithm, and choosing mechanisms

(a) Shor's algorithm turns the ECDLP into a *period-finding* problem. For $Q = kP$ in a group of order n, define $f(x, y) = xP + yQ = (x + ky)P$. Then $f(x, y) = f(x', y')$ exactly when $(x - x') \equiv -k(y - y') \pmod{n}$, so f is periodic with the hidden period lattice generated by $(k, -1)$ and $(n, 0)$. A quantum computer evaluates f on a superposition of all (x, y) (the costly part: reversible point addition and modular inversion), then applies the quantum Fourier transform; measurement yields pairs (u, v) with $u \equiv kv \pmod{n}$, from which $k = u\, v^{-1} \bmod n$. The cost is polynomial, about $O(\ell^3)$ gates for an ℓ-bit curve. *Grover's* algorithm only gives a quadratic speed-up for unstructured search: brute-forcing a 256-bit key would take $\approx 2^{128}$ quantum steps, which is no better than Pollard rho classically, and Grover does not parallelise well. So Grover is a concern for symmetric keys (and is handled by AES-256), while Shor completely breaks ECC.

Resource estimates. Litinski (2023) estimates that a 256-bit elliptic-curve private key can be computed with only about **50 million Toffoli gates**; on his assumed architecture (6 000 modules of 1 152 physical qubits each) one key would be broken about every 10 minutes [17]. Earlier work by Roetteler et al. estimated about 2 330 logical qubits for P-256 [18]. For RSA-2048, Gidney (2025) estimates that factoring is possible with **fewer than one million noisy qubits in less than a week**, using about **6.5 billion Toffoli gates** and roughly 1 400–1 550 logical qubits [19]. ECC may fall first because the quantum cost depends on the size of the numbers, not on the classical security: Shor on a 256-bit curve operates on 256-bit numbers, while RSA-2048 needs arithmetic on 2048-bit numbers. The ECC circuit therefore needs a similar number of logical qubits but about *100 times fewer* gates ($5 \cdot 10^7$ versus $6.5 \cdot 10^9$) and much less time. The very property that makes ECC efficient classically (short keys) makes it a smaller target for a quantum computer.

(b) In a "harvest now, decrypt later" (HNDL) attack an adversary records encrypted traffic today (including the ECDHE key shares) and stores it until a cryptographically relevant quantum computer exists; Shor then recovers the ephemeral private keys, the session keys and the plaintext. This is urgent for **ECDH** because the damage is retroactive: data that must stay confidential for 10–20 years (health, government, trade secrets) is already at risk, and forward secrecy does not help against Shor. For **signatures**, the attacker needs the quantum computer *at the time of verification* to forge: a signature that was verified in a 2026 handshake cannot be forged retroactively, so migration can wait until shortly before quantum computers arrive. *Exceptions* that need early migration: signatures that must remain verifiable for a long time or roots of trust that cannot be changed later, e.g. **firmware/secure-boot keys burned into devices** with a 10–20-year lifetime, long-lived root CA certificates, and legally binding documents or code signatures that must remain trustworthy for decades (an attacker could later forge a back-dated document). NIST IR 8547 (draft) plans to deprecate quantum-vulnerable algorithms at 112-bit security by 2030 and disallow all of them by 2035 [20].

(c) Design scenario: MedLink (launch 2027, 20-year retention).

Requirement	Mechanism and parameters	PQ / hybrid now?	Justification
R1. Secure channel app ↔ server	TLS 1.3 with hybrid key exchange `X25519MLKEM768` (X25519 + ML-KEM-768, FIPS 203 [21]); AES-256-GCM; server authentication with ECDSA P-256 certificate; certificate pinning in the app	**Yes** (KEM); **plan** (signature)	Patient data must stay confidential for 20 years, so HNDL applies: the key exchange must be post-quantum now. The hybrid keeps X25519 security if ML-KEM is broken. Authentication is checked online, so the certificate can migrate to ML-DSA later; the app is updatable.
R2. Prescriptions verifiable for 10 years	Composite/dual signature ECDSA P-256 (or Ed25519) + ML-DSA-65 (FIPS 204 [22]), with RFC 3161 time-stamps and long-term validation	**Yes**	Signatures must be valid until ≥ 2037, after the planned 2035 disallowance of ECC; a forged prescription later would be a legal and safety problem. Hybrid gives legal acceptance today (ECDSA) and quantum resistance (ML-DSA). Alternative: SLH-DSA (FIPS 205 [23]) for maximum conservatism.
R3. Encrypted record export to offline recipient	HPKE (RFC 9180) with a hybrid KEM X25519 + ML-KEM-768 (e.g. X-Wing), HKDF-SHA256, AES-256-GCM; record signed by MedLink	**Yes**	Offline recipient ⇒ no interactive handshake, so a one-shot KEM + AEAD is needed. The file can be stored and copied for decades, so it is the classic HNDL target. AES-256 also covers Grover.
R4. Firmware updates for a 10-year blood-pressure monitor	Firmware signed with a stateful hash-based signature LMS/HSS (NIST SP 800-208 [24]), optionally in parallel with Ed25519; verification key in boot ROM; anti-rollback counter	**Yes**	The verification key is fixed in hardware for 10 years and cannot be changed after deployment. LMS verification is only hashing (cheap for a low-power MCU) and its security relies only on SHA-256. State management is easy because MedLink signs few firmware images in an HSM.

Conclusion

ECC is the backbone of modern secure communication: the ECDLP has no known sub-exponential classical attack, so 256-bit keys provide 128-bit security with small keys, fast signing and cheap key exchange, which explains its use in TLS 1.3, Signal, SSH and cryptocurrencies. The calculations on the toy curve showed why computing kP is easy and recovering k is hard only because n is huge. The case studies show, however, that real-world failures almost never come from the mathematics: reused or biased nonces (Sony, PuTTY), weak random number generators, side channels and faulty certificate validation broke real systems while the ECDLP remained intact. Correct, constant-time, deterministic implementations and named, vetted curves are therefore as important as the choice of the curve itself. Furthermore, ECC alone does not provide authentication of key exchanges; certificates, signatures or identity verification are required to stop man-in-the-middle attacks. Its biggest limitation is the quantum threat: Shor's algorithm breaks ECC in polynomial time, probably with fewer resources than RSA, and recorded ECDH traffic is already exposed to harvest-now-decrypt-later. The right path for the coming decade is hybrid cryptography (X25519 + ML-KEM, ECDSA + ML-DSA) and crypto-agility, so that ECC is kept as a well-understood safety net while post-quantum algorithms mature.

Appendix A — Python Verification: Chosen Values and Outputs

The three scripts are in the repository https://github.com/Diellart-Mulolli/ecc-lab (see its README for the running guide). They were run with Python 3.12.3 and the `cryptography` package 41.0.7 (OpenSSL backend). The outputs below are copied unchanged from the terminal. In addition, every value was cross-checked independently: the toy-curve results by a second computation along a different path, and the P-256 values with OpenSSL (through `cryptography`) and the system `sha256sum` tool.

A.1 `q1_toy_curve.py` (Question 1)

Chosen value	Setting in the code
Curve	$E : y^2 \equiv x^3 + ax + b \pmod p$ with $p = 5$, $a = 2$, $b = 1$ (lab handout, Stamp Sec. 4.5.1)
Base point and public key	$P = (0, 1)$, $Q = (3, 2)$
Point at infinity	represented by `None`, printed as `inf`
Group law	slope formulas of Stamp, Table 4.1; inverses with `pow(x, -1, p)`
Method for (b)	exhaustive search: add P repeatedly until the result equals Q

Command: `python3 q1_toy_curve.py`

```
Affine points: [(0, 1), (0, 4), (1, 2), (1, 3), (3, 2), (3, 3)] + infinity; #E = 7

(a)
  (0, 1) + (0, 1) (doubling): m = 2 * 2^-1 = 1, x3 = 1^2 - 0 - 0 = 1, y3 = 1(0 - 1) - 1 = 3
  2P = (1, 3) on curve: True
  (1, 3) + (0, 1) (addition): m = 3 * 4^-1 = 2, x3 = 2^2 - 1 - 0 = 3, y3 = 2(1 - 3) - 3 = 3
  3P = (3, 3) on curve: True

(b) brute force for Q = (3, 2)
  2P = (1, 3)
  3P = (3, 3)
  4P = (3, 2)
  k = 4, found after 3 point additions/doublings

All multiples of P:
  1P = (0, 1)
  2P = (1, 3)
  3P = (3, 3)
  4P = (3, 2)
  5P = (1, 2)
  6P = (0, 4)
  7P = inf
  8P = (0, 1)
```

Check. The six affine points match the set in the lab handout, and $\#E = 7$. The slopes $m = 1$ and $m = 2$, the points $2P = (1, 3)$ and $3P = (3, 3)$, and $k = 4$ after 3 operations are the hand results of Question 1(a)–(b). $7P = \infty$ confirms that P has order 7. Independent cross-check: computing $4P$ as the doubling $2(2P)$ instead of $3P + P$ also gives $(3, 2)$, and counting points with Euler's criterion also gives $\#E = 7$.

A.2 `q2_benchmark.py` (Question 2(c))

Chosen value	Setting in the code
Signed message	`"sensor reading: 21.4 C, 48 % RH"` (31 bytes)
RSA	3072-bit modulus, $e = 65537$, RSA-PSS with MGF1-SHA-256, 32-byte salt, SHA-256
ECDSA	curve P-256 (`secp256r1`), SHA-256, DER-encoded signature
EdDSA	Ed25519 (RFC 8032, SHA-512 internally)
Repetitions	key generation: 5 for RSA, 500 for ECDSA and Ed25519; signing and verification: 300 each
Measurement	`time.perf_counter()`, mean time per operation in ms; key sizes as X.509 DER (`SubjectPublicKeyInfo`)

Command: `python3 q2_benchmark.py` (the run used for Table 2)

```
scheme            keygen ms   sign ms  verify ms  pk DER B  sig B
RSA-3072 (PSS)      813.091     1.386      0.092       422    384
ECDSA P-256           0.058     0.047      0.114        91     72
Ed25519               0.067     0.060      0.155        44     64

P-256 public key: uncompressed 65 B, compressed 33 B
Ed25519 public key raw: 32 B
```

Check. Table 2 rounds these numbers. Timings change from run to run (three runs gave RSA key generation times of 356, 813 and 1086 ms), but the order is always the same: RSA is slowest at key generation and signing and fastest at verification. Derived values: $813.091/0.058 \approx 14\,000$ (four orders of magnitude), $1.386/0.047 \approx 29$ and $1.386/0.060 \approx 23$ (signing $\approx 25\times$ slower), and $50\,000/60 \cdot 0.155$ ms ≈ 0.13 s of CPU time per second ($\approx 13\%$). The sizes were confirmed separately: RSA-3072 modulus and signature 384 B, DER public keys 422 B (RSA), 91 B (P-256) and 44 B (Ed25519). DER-encoded ECDSA signatures varied between 69 and 72 B over 200 signatures, because DER drops leading zero bytes of r and s and adds a sign byte when the top bit is set.

A.3 `q3_nonce_reuse.py` (Question 3(c))

Chosen value	Setting in the code
Curve	NIST P-256 [2], [6]: $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$, $a = -3$
p	`0xffffffff00000001000000000000000000000000ffffffffffffffffffffffff`
order n	`0xffffffff00000000ffffffffffffffffbce6faada7179e84f3b9cac2fc632551`
G_x	`0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c296`
G_y	`0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f5`
Hash	$h = \text{SHA-256}(m)$ read as an integer, reduced mod n
Messages	$m_1 =$ `"firmware 3.55"`, $m_2 =$ `"homebrew loader"`
Private key d, nonce k	uniform in $[1, n-1]$ from Python's `random.Random(2026)` (`--seed 2026`, for reproducibility only; without the option the script uses the cryptographic `secrets` module)
Scalar multiplication	left-to-right double-and-add (demonstration only, not constant-time)
Forgery	message `"anything the attacker wants"` signed with the recovered d and nonce 12345

Command: `python3 q3_nonce_reuse.py --seed 2026`

```
victim private key d = 0xf3f49249dc28ff90a5aec7978306d03bf38b2ffc80a4df5a51c9bc701e7ea41a
reused nonce       k = 0x6bad6be28e7aa6e99f19950499dd251de512148239292d22e255accb1a466885
signature 1 on 'firmware 3.55':
  h1 = 0xba2c60557cd36927df60802fa63d01ff7bccb978daee9847b797aedbafcd7bbc
  r1 = 0x45f4600c8a058ddaea76061d9eb761b67b7799eb14fbf231d21bf31f95c60b8e
  s1 = 0xa22af549b244c43a15a430634a4d87ff55eb0dcbc0ba4c7bd8e1eaa53b1ea7b8
signature 2 on 'homebrew loader':
  h2 = 0xaabb15092f5617c4891d2a5517315f2069371074a2af28a33484e2c0de2576b2
  r2 = 0x45f4600c8a058ddaea76061d9eb761b67b7799eb14fbf231d21bf31f95c60b8e
  s2 = 0x33ccd118c13f82ff4ed22302c5f1b2477997f28853e00b7295e2aca1da1edb11
both signatures valid: True True
r1 == r2 (visible reuse): True
recovered k = 0x6bad6be28e7aa6e99f19950499dd251de512148239292d22e255accb1a466885
recovered d = 0xf3f49249dc28ff90a5aec7978306d03bf38b2ffc80a4df5a51c9bc701e7ea41a
recovered k correct: True
recovered d correct: True
forged signature verifies: True
```

Check. $r_1 = r_2$ exposes the reused nonce. Inserting the printed values into the formulas of Question 3(c):

$$\begin{aligned} k &= (h_1 - h_2)(s_1 - s_2)^{-1} \bmod n \\ &= \texttt{0x6bad6be28e7aa6e99f19950499dd251de512148239292d22e255accb1a466885}, \\ d &= (s_1 k - h_1)\, r^{-1} \bmod n \\ &= \texttt{0xf3f49249dc28ff90a5aec7978306d03bf38b2ffc80a4df5a51c9bc701e7ea41a}, \end{aligned}$$

which are exactly the nonce and private key chosen by the script. Independent cross-checks with OpenSSL: the generator G in the script equals OpenSSL's P-256 generator; $x(kG) \bmod n$ computed by OpenSSL equals r; both signatures are accepted by OpenSSL's ECDSA verification under $Q = dG$; and h_1, h_2 equal the `sha256sum` digests of the two messages.

References

[1] M. Stamp, *Information Security: Principles and Practice*, 3rd ed. Hoboken, NJ, USA: Wiley, 2022.

[2] L. Chen, D. Moody, K. Randall, A. Regenscheid, and A. Robinson, "Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-186, Feb. 2023, doi: 10.6028/NIST.SP.800-186.

[3] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," Internet Eng. Task Force, RFC 7748, Jan. 2016. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7748

[4] E. Barker, "Recommendation for key management: Part 1 – General," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-57 Part 1 Rev. 5, May 2020, doi: 10.6028/NIST.SP.800-57pt1r5.

[5] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," Internet Eng. Task Force, RFC 8032, Jan. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8032

[6] *Digital Signature Standard (DSS)*, FIPS PUB 186-5, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Feb. 2023, doi: 10.6028/NIST.FIPS.186-5.

[7] R. Barnes, K. Bhargavan, B. Lipp, and C. Wood, "Hybrid public key encryption," Internet Eng. Task Force, RFC 9180, Feb. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9180

[8] fail0verflow, "Console hacking 2010: PS3 epic fail," presented at the 27th Chaos Commun. Congr. (27C3), Berlin, Germany, Dec. 2010.

[9] T. Pornin, "Deterministic usage of the digital signature algorithm (DSA) and elliptic curve digital signature algorithm (ECDSA)," Internet Eng. Task Force, RFC 6979, Aug. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6979

[10] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," Internet Eng. Task Force, RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

[11] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," Internet Eng. Task Force, RFC 5869, May 2010. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5869

[12] M. Marlinspike and T. Perrin, "The X3DH key agreement protocol," Signal Messenger, Nov. 2016. Accessed: Oct. 13, 2026. [Online]. Available: https://signal.org/docs/specifications/x3dh/

[13] E. Kret and R. Schmidt, "The PQXDH key agreement protocol," Signal Messenger, 2023. Accessed: Oct. 13, 2026. [Online]. Available: https://signal.org/docs/specifications/pqxdh/

[14] S. Tatham, "PuTTY vulnerability vuln-p521-bias," PuTTY wishlist, Apr. 2024. Accessed: Oct. 13, 2026. [Online]. Available: https://www.chiark.greenend.org.uk/~sgtatham/putty/wishlist/vuln-p521-bias.html

[15] F. Bäumer and M. Brinkmann, "CVE-2024-31497: Secret key recovery of NIST P-521 private keys through biased ECDSA nonces in PuTTY client," oss-security mailing list, Apr. 15, 2024. Accessed: Oct. 13, 2026. [Online]. Available: https://www.openwall.com/lists/oss-security/2024/04/15/6

[16] Nat. Inst. Standards Technol., "CVE-2024-31497 detail," National Vulnerability Database. Accessed: Oct. 13, 2026. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2024-31497

[17] D. Litinski, "How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates," 2023, *arXiv:2306.08585*.

[18] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, "Quantum resource estimates for computing elliptic curve discrete logarithms," in *Advances in Cryptology – ASIACRYPT 2017* (Lecture Notes in Computer Science, vol. 10625). Cham, Switzerland: Springer, 2017, pp. 241–270, doi: 10.1007/978-3-319-70697-9_9.

[19] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," 2025, *arXiv:2505.15917*.

[20] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, "Transition to post-quantum cryptography standards," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST IR 8547 (Initial Public Draft), Nov. 2024, doi: 10.6028/NIST.IR.8547.ipd.

[21] *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS PUB 203, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Aug. 2024, doi: 10.6028/NIST.FIPS.203.

[22] *Module-Lattice-Based Digital Signature Standard*, FIPS PUB 204, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Aug. 2024, doi: 10.6028/NIST.FIPS.204.

[23] *Stateless Hash-Based Digital Signature Standard*, FIPS PUB 205, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Aug. 2024, doi: 10.6028/NIST.FIPS.205.

[24] D. A. Cooper, D. C. Apon, Q. H. Dang, M. S. Davidson, M. J. Dworkin, and C. A. Miller, "Recommendation for stateful hash-based signature schemes," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST SP 800-208, Oct. 2020, doi: 10.6028/NIST.SP.800-208.